

NutriDeby

Plano de Desenvolvimento — Fase 0 (MVP Vendável)

SaaS B2B2C para nutricionistas (Controladora, assinante) e pacientes (Titular, uso gratuito via PWA). Operado pela WSS+13.

Documento	Plano de desenvolvimento consolidado
Fase	Fase 0 — remediação + MVP vendável
Emissão	29/08/2026
Base	Backlog Fase 0, arquitetura técnica e estado real do repositório de rebuild

Documento interno de engenharia e produto. Contém decisões de arquitetura, estado de implementação e roadmap. Não é peça de captação.

1. Contexto e Objetivo

O projeto foi consolidado a partir de uma decisão estratégica de **pausar o NutriVerse** e concentrar toda a engenharia em um único sistema, herdando a stack já validada em produção do NutriDeby. A avaliação de especialistas convergiu que manter dois produtos com stacks e modelos de negócio distintos, em operação solo (fundadora + Claude Code), fragmenta o foco onde ele é mais escasso.

Objetivo da Fase 0: entregar um MVP *vendável* — a nutricionista se cadastra, assina um plano, paga via Asaas, atende pacientes com prontuário e plano alimentar, e o paciente acessa tudo por um PWA instalável. O diferencial de IA (Agente Clínico RAG) entra como recurso de plano superior, sempre sob aprovação humana.

Princípio estrutural inegociável: nenhuma saída de IA chega ao paciente sem aprovação explícita da nutricionista. A fila de aprovação é tabela central do sistema, não recurso opcional.

2. Escopo do MVP

Dentro do escopo	Fora do escopo (adiado)
• PWA + Web para o paciente (não app nativo) • Web Admin para a nutricionista (prontuário, fila de aprovação, cobrança) • Multi-tenant desde a primeira linha • Pagamento recorrente (Asaas) • Agente Clínico RAG com gate de aprovação	• App nativo iOS/Android • Múltiplos agentes de IA nomeados • Kubernetes, Neo4j, HeyGen • WhatsApp (Telegram é o canal desta fase) • Captação externa antes de validação

3. Arquitetura e Stack

Monorepo **poliglota**, sem workspace unificado: cada pacote tem seu próprio `package.json/package-lock.json` e é instalado, buildado e deployado de forma independente (npm). O `rag-agent` é Python (FastAPI), fora do ecossistema Node. Decisão de arquitetura registrada: **não** converter para pnpm/Turborepo — os deploys per-pacote (Railway com `--path-as-root`; Vercel com `root = diretório do app`) dependem do lockfile local, que um workspace hoisted quebraria.

Componente	Stack	Hospedagem
apps/admin-web	Next.js + TS + Tailwind + shadcn/ui	Vercel
apps/pwa-patient	Vite + React + TS + vite-plugin-pwa	Vercel
services/api-gateway	NestJS + Prisma + PostgreSQL	Railway (produção)
services/rag-agent	FastAPI + pgvector + Claude + Voyage	Local (a hospedar)
services/telegram-bot	Node + TS (grammY)	Local (a hospedar)

4. Governança de Compliance (não-negociável)

- **Gate de aprovação humana:** nenhum plano/rascunho de IA chega ao paciente sem `aprovadoPeloNutri=true`. Rotas `/me/*` filtram aprovação.
- **Disclaimer CFN:** todo output de IA clínica carrega “sugestão para revisão do profissional”.
- **Consentimento:** Termo obrigatório antes de qualquer tratamento de dado de saúde; revogação respeitada.
- **Auditoria:** toda operação sobre dado sensível é registrada (AuditLog centralizado).
- **Criptografia:** AES-256 em repouso, TLS 1.2+ em trânsito; nenhum dado de saúde em URL ou log em texto plano.

Estas regras são reforçadas no repositório por uma skill de checklist (compliance-guard) e por um agente revisor independente (compliance-reviewer) executado antes de merges sensíveis.

5. Estado Atual de Implementação

Situação real do repositório de rebuild. O api-gateway está em produção no Railway (Postgres gerenciado, schema aplicado, base TACO com 597 alimentos seedada) e o PWA do paciente já consome esse backend real. O **Web Admin permanece mockado**. Legenda: **Concluído** = entregue e testado ponta a ponta; **Parcial** = backend pronto mas UI/host pendente; **Pendente** = não iniciado ou bloqueado.

#	Item do backlog (Fase 0)	Status
1	Aditivo de DPA / Política de Privacidade (Telegram como subprocessador)	Pendente
2	Infra base: Postgres+pgvector, ambientes, CI/CD	Parcial
3	Cadastro de nutricionista + validação de CRN	Parcial
4	Cadastro de paciente + Termo de Consentimento	Concluído
5	Criptografia em repouso/trânsito + logs de auditoria	Concluído
6	Prontuário: anamnese + antropometria	Parcial
7	Gráfico de evolução antropométrica	Parcial
8	Cálculo nutricional (base TACO)	Concluído
9	Construtor de plano alimentar + duplicação	Parcial
10	Revogação de consentimento (fluxo LGPD)	Concluído
11	Integração Asaas: assinatura recorrente (Pix/cartão)	Concluído
12	Estrutura de planos com limite por plano	Concluído
13	Histórico de faturas + recibo em PDF	Parcial
14	Suspensão de acesso por inadimplência (com tolerância)	Concluído
15	PWA instalável: login + visualizar plano	Concluído
16	Diário alimentar simples (texto/foto sem IA)	Concluído
17	Registro de peso pelo paciente	Concluído
18	Bot Telegram: lembretes e notificações	Parcial
19	Adaptador de Canais (abstração p/ WhatsApp futuro)	Parcial
20	Agente Clínico RAG: rascunho de plano por IA	Concluído
21	Fluxo de aprovação obrigatória + disclaimer CFN	Concluído
22	Gate de feature por plano de assinatura	Concluído

Resumo: **13 concluídos**, 8 parciais, 1 pendente — de 22 itens. O núcleo de monetização e o fluxo do paciente estão funcionais; o gargalo é o Web Admin (UI real) e os bloqueios de hospedagem/DPA.

6. Roadmap por Sprints

Sprint	Bloco	Foco
1–2	Fundação e Compliance	Infra, cadastro, consentimento, criptografia, auditoria, DPA
3–4	Core de Valor (Admin Web)	Prontuário, antropometria, cálculo TACO, construtor de plano
5–6	Monetização	Asaas, planos e limites, faturas, suspensão por inadimplência
7–8	PWA Paciente + Telegram	PWA, diário, peso, bot de lembretes, adaptador de canais
9–10	Diferencial de IA (Pro)	Agente RAG, aprovação + disclaimer, gate por plano

7. Planos de Assinatura

Plano	Preço/mês	Pacientes	Recursos
Starter	R\$ 49	até 15	Admin Web, PWA, prontuário, plano manual, Telegram (lembretes)
Pro	R\$ 129	até 60	Tudo do Starter + Agente Clínico RAG + agenda Google
Clínica	R\$ 299	ilimitado	Tudo do Pro + multiusuário + faturamento consolidado

Trial de 14 dias em qualquer plano, sem cartão no cadastro; cobrança inicia ao fim do trial com aviso 2 dias antes. IA reservada ao Pro para não subsidiar custo de tokens no plano de entrada.

8. Riscos e Bloqueios Ativos

Bloqueio	Natureza	Impacto / Mitigação
Aditivo de DPA (Telegram/e-mail)	Jurídico, não técnico	Bloqueia notificação real ao paciente em produção. Minutas prontas (Telegram e e-mail); decisão adiada.
Web Admin mockado	Produto	Nutricionista ainda não opera com dados reais na UI. Próximo maior valor de desenvolvimento.
rag-agent e telegram-bot locais	Infra	Rodam só localmente. Falta hospedar (definir RAG_AGENT_BASE_URL no gateway).
Tipos de API duplicados	Arquitetura	Contrato replicado entre back e front. Mitigação proposta: codegen a partir do schema Prisma.

9. Próximos Passos Priorizados

- 1. Tirar o Web Admin do mock** — Maior valor de produto; onde o revisor de compliance passa a ser essencial (admin vê dado de saúde).
- 2. Fonte única de tipos via codegen** — Gerar contrato a partir do schema Prisma, sem quebrar os deploys per-pacote.
- 3. Hospedar rag-agent e telegram-bot** — Sair do local; configurar RAG_AGENT_BASE_URL e host do bot.
- 4. Fechar decisão de canal + DPA** — Telegram vs. e-mail; destrava a notificação real ao paciente.

10. Definition of Done — Fase 0

- Nutricionista cadastra, assina um plano e paga via Asaas
- Paciente assina consentimento e acessa o PWA
- Plano alimentar montado e enviado com cálculo nutricional correto

- Lembretes funcionando via canal aprovado
- DPA / Política de Privacidade atualizados e vigentes
- Auditoria de segurança básica (criptografia, logs) validada